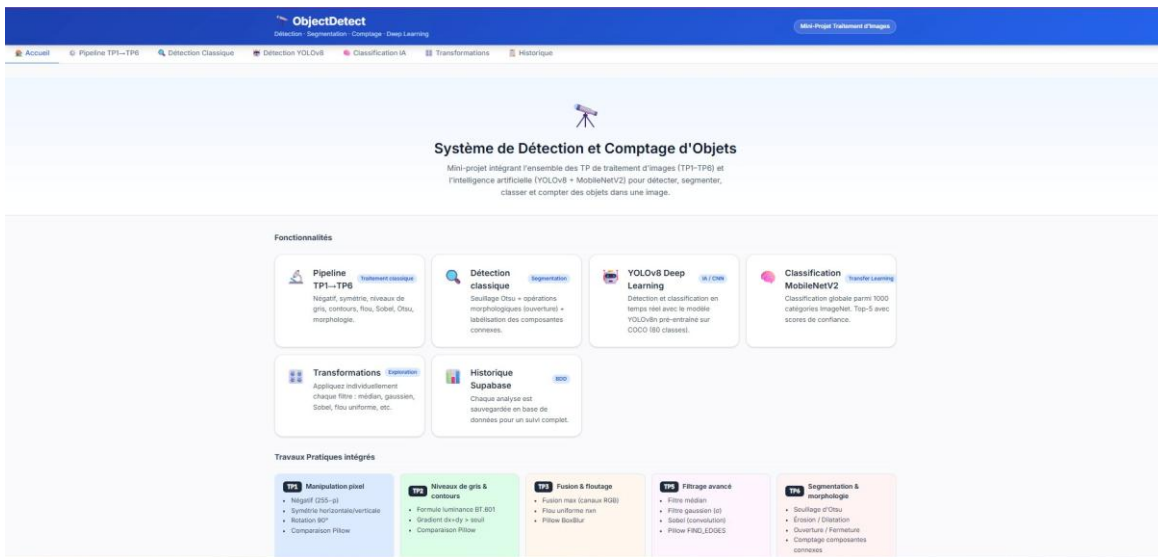


# MINI-PROJET

# Traitement d'Images

ObjectDetect — Système de Détection, Segmentation & Classification



Interface principale — Accueil de ObjectDetect

|                 |                                                   |
|-----------------|---------------------------------------------------|
| Projet          | Mini-Projet Traitement d'Images (TP1–TP6)         |
| Framework       | React + TypeScript (Vite) / FastAPI (Python)      |
| IA intégrée     | YOLOv8 Deep Learning + MobileNetV2 Classification |
| Base de données | Supabase (Historique des analyses)                |
| Date            | Avril 2026                                        |

# 1. Introduction et Présentation du Projet

ObjectDetect est une application web full-stack conçue dans le cadre du mini-projet de traitement d'images. Elle intègre l'ensemble des travaux pratiques (TP1 à TP6) couvrant les techniques classiques de traitement d'images ainsi que l'intelligence artificielle moderne (YOLOv8 et MobileNetV2) pour détecter, segmenter, classer et compter des objets dans une image.

Le projet est structuré en deux parties principales :

- Frontend : React + TypeScript avec Vite, interface utilisateur moderne et responsive
- Backend : FastAPI (Python) avec OpenCV, Pillow, Ultralytics YOLOv8 et TensorFlow
- Base de données : Supabase pour la persistance de l'historique des analyses

## 1.1 Structure du Projet

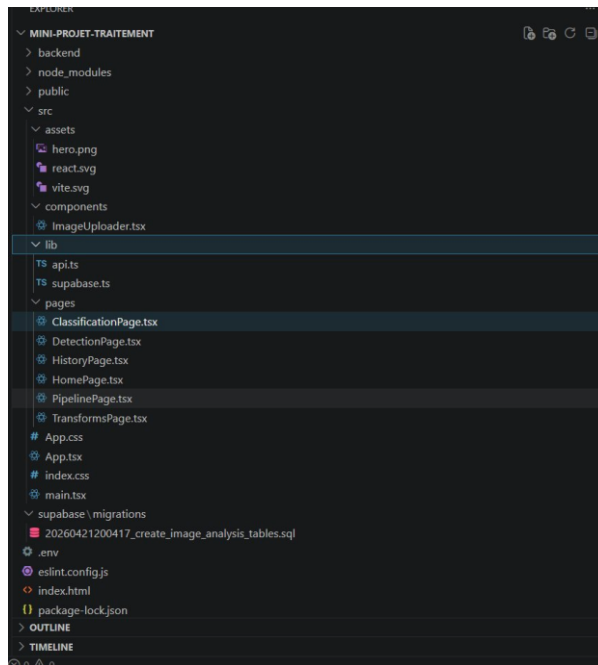


Fig. 1 — Structure des fichiers (partie 1)

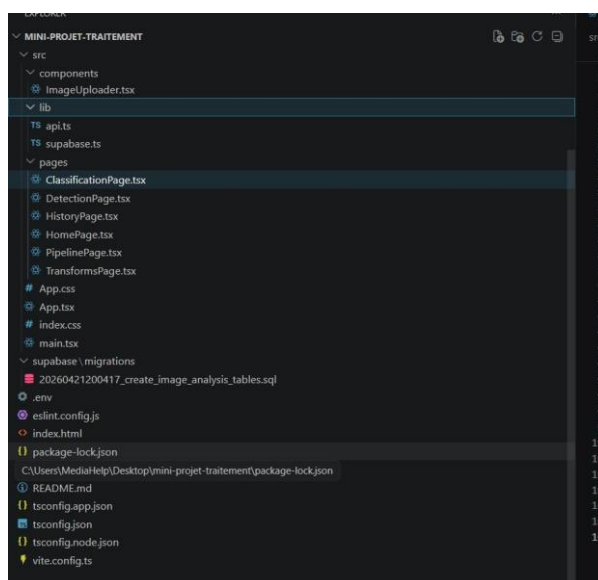


Fig. 2 — Structure des fichiers (partie 2)

La structure du projet suit une architecture claire :

```
mini-projet-traitement/  
├── backend/                ← API FastAPI Python  
│   └── main.py             ← Point d'entrée du serveur  
├── src/  
│   ├── components/  
│   │   └── ImageUploader.tsx  
│   ├── lib/  
│   │   ├── api.ts         ← Appels API frontend  
│   │   └── supabase.ts  
│   ├── pages/  
│   │   ├── ClassificationPage.tsx  
│   │   ├── DetectionPage.tsx  
│   │   ├── HistoryPage.tsx  
│   │   ├── HomePage.tsx  
│   │   ├── PipelinePage.tsx  
│   │   └── TransformsPage.tsx  
│   └── App.tsx  
└── supabase/migrations/
```

## 2. Architecture Technique

## 2.1 Frontend — api.ts

Le fichier `src/lib/api.ts` centralise tous les appels HTTP vers le backend FastAPI. Il utilise l'URL configurée via la variable d'environnement `VITE_API_URL` (par défaut `http://localhost:8000`).

```
// src/lib/api.ts
```

```
const API_BASE = import.meta.env.VITE_API_URL || 'http://localhost:8000';

Tabnine | Edit | Test | Explain | Document | Windsurf: Refactor | Explain | Generate JSDoc | X
export async function runPipeline(file: File) {
  const form = new FormData();
  form.append('file', file);
  const res = await fetch(`${API_BASE}/api/pipeline`, { method: 'POST', body: form });
  if (!res.ok) throw new Error((await res.json()).detail || 'Pipeline error');
  return res.json();
}

Tabnine | Edit | Test | Explain | Document | Windsurf: Refactor | Explain | Generate JSDoc | X
export async function detectClassical(file: File, minArea = 100) {
  const form = new FormData();
  form.append('file', file);
  form.append('min_area', String(minArea));
  const res = await fetch(`${API_BASE}/api/detect/classical`, { method: 'POST', body: form });
  if (!res.ok) throw new Error((await res.json()).detail || 'Detection error');
  return res.json();
}

Tabnine | Edit | Test | Explain | Document | Windsurf: Refactor | Explain | Generate JSDoc | X
export async function detectYolo(file: File, conf = 0.25) {
  const form = new FormData();
  form.append('file', file);
  form.append('conf', String(conf));
  const res = await fetch(`${API_BASE}/api/detect/yolo`, { method: 'POST', body: form });
  if (!res.ok) throw new Error((await res.json()).detail || 'YOLO error');
  return res.json();
}

Tabnine | Edit | Test | Explain | Document | Windsurf: Refactor | Explain | Generate JSDoc | X
export async function classifyImage(file: File) {
  const form = new FormData();
  form.append('file', file);
  const res = await fetch(`${API_BASE}/api/classify`, { method: 'POST', body: form });
  if (!res.ok) throw new Error((await res.json()).detail || 'Classification error');
  return res.json();
}

Tabnine | Edit | Test | Explain | Document | Windsurf: Refactor | Explain | Generate JSDoc | X
export async function applyTransform(file: File, operation: string, param = 1.5) {
  const form = new FormData();
  form.append('file', file);
  form.append('operation', operation);
  form.append('param', String(param));
  const res = await fetch(`${API_BASE}/api/transform`, { method: 'POST', body: form });
  if (!res.ok) throw new Error((await res.json()).detail || 'Transform error');
  return res.json();
}

Tabnine | Edit | Test | Explain | Document | Windsurf: Refactor | Explain | Generate JSDoc | X
export function b64ToUrl(b64: string): string {
  return `data:image/png;base64,${b64}`;
}
```

## 2.2 Configuration Environnement

Le fichier `.env` à la racine du projet configure l'URL du backend :

[illegible]

Le lancement du projet requiert deux terminaux simultanés :

```
C:\Users\MediaHelp\Desktop\mini-projet-traitement\backend>cd C:\Users\MediaHelp\Desktop\mini-projet-traitement\backend
C:\Users\MediaHelp\Desktop\mini-projet-traitement\backend>uvicorn main:app --reload --port 8000
INFO: Will watch for changes in these directories: ['C:\\Users\\MediaHelp\\Desktop\\mini-projet-traitement\\backend']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

```
PS C:\Users\MediaHelp> cd C:\Users\MediaHelp\Desktop\mini-projet-traitement
PS C:\Users\MediaHelp\Desktop\mini-projet-traitement> npm run dev

> project@0.0.0 dev
> vite

VITE v8.0.9 ready in 184 ms
  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help
```

[illegible]

```

    /* TP Recap */
    <div className="section">
      <div className="section-title">Travaux Pratiques intégrés</div>
      <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(220px, 1fr))', gap: 12 }}>
        {TPs.map((tp) => (
          <div key={tp.id} style={{ background: tp.color, border: '1px solid var(--neutral-200)', borderRadius: 'var(--radius-md)', padding: '14px 16px' }}>
            <div style={{ display: 'flex', alignItems: 'center', gap: 8, marginBottom: 8 }}>
              <span style={{ background: 'var(--neutral-800)', color: 'white', borderRadius: 4, padding: '2px 8px', fontSize: '0.72rem', fontFamily: 'var(--font-mono)', fontWeight: 700 }}{tp.id}</span>
              <span style={{ fontSize: '0.82rem', fontWeight: 600, color: 'var(--neutral-800)' }}>{tp.title}</span>
            </div>
            <ol style={{ paddingLeft: 16, fontSize: '0.78rem', color: 'var(--neutral-700)', lineHeight: 1.6 }}>
              {tp.items.map((item) => <li key={item}>{item}</li>)}
            </ol>
          </div>
        ))}
      </div>
    </div>

    /* Architecture */
    <div className="section">
      <div className="section-title">Architecture technique</div>
      <div className="card">
        <div style={{ display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(180px, 1fr))', gap: 14, fontSize: '0.82rem' }}>
          {
            [
              { layer: 'Frontend', items: ['React 18 + TypeScript', 'Vite', 'Supabase JS'] },
              { layer: 'Backend API', items: ['Python + FastAPI', 'Postgres / Prisma', 'NumPy + SciPy'] },
              { layer: 'Deep Learning', items: ['PyTorch', 'Ultralytics YOLOv8', 'MolScribe (torchvision)'] },
              { layer: 'Base de données', items: ['Supabase (PostgreSQL)', 'RLS activé', 'Historique des analyses'] },
            ].map((col) => (
              <div key={col.layer}>
                <div style={{ fontweight: 600, color: 'var(--neutral-700)', marginBottom: 6, fontSize: '0.85rem' }}>{col.layer}</div>
                <ol style={{ paddingLeft: 14, color: 'var(--neutral-500)', lineHeight: 1.8 }}>
                  {col.items.map((i) => <li key={i}>{i}</li>)}
                </ol>
              </div>
            ))}
        </div>
      </div>
    </div>
  </div>
)
}

```

## 3.2 Pipeline de Traitement TP1→TP6

La page Pipeline permet d'exécuter l'ensemble de la chaîne de traitement d'images en une seule opération. Elle applique successivement les transformations TP1 à TP6 et affiche les 10 résultats intermédiaires.

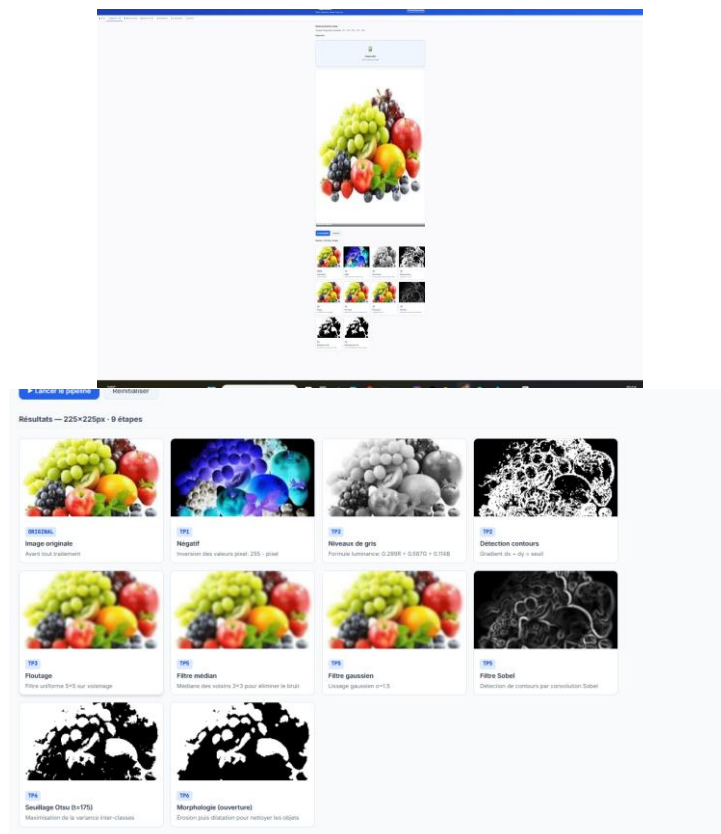


Fig. 4 — Pipeline complet : 10 étapes de traitement affichées

Code source — PipelinePage.tsx :

```
// src/pages/PipelinePage.tsx
```

```
export { useState } from 'react';
import { runPipeline, loadPipe } from './utils/api';
import { Component, PageLoader };

WidgetRefactor.Explain
interface Step {
  name: string;
  stage: string;
  description: string;
}

WidgetRefactor.Explain
interface PipelineResult {
  original: string;
  steps: Step[];
  width: number;
  height: number;
}

Tabview [Edit | Test] Explain | Document | WidgetRefactor | Explain | Generate SDoc >
export default function PipelinePage() {
  const [file, setFile] = useState<File | null>(null);
  const [preview, setPreview] = useState<string | null>(null);
  const [loading, setLoading] = useState<false>;
  const [result, setResult] = useState<PipelineResult | null>(null);
  const [error, setError] = useState<string | null>(null);

  WidgetRefactor.Explain | Generate SDoc >
  async function handleRun() {
    if (!file) return;
    setLoading(true);
    setError(null);
    try {
      const data = await runPipeline(file);
      setResult(data);
    } catch (e: unknown) {
      setError instanceof Error ? e.message : "Error incoming";
    } finally {
      setLoading(false);
    }
  }

  WidgetRefactor.Explain | Generate SDoc >
  function toggleFile(event: React.MouseEvent) {
    const e = new MouseEvent("click");
    return e.button === 0 ? "TP";
  }

  return (
    <div>
      <div className="section">
        <div className="card-title">Pipeline de traitement complet.</div>
        <p style={loading ? {} : margin:bottom: 16}></p>
        Visualiser chaque étape de Traitement : TP1 + TP2 + TP3 + TP4 + TP5.
        <p></p>
        <Image alt="Diagramme de la pipeline de traitement" />
        <button onClick={() => setFile(file)}> Sélectionner le fichier </button>
        <button onClick={() => preview(loadPipe(preview))}> Visualiser l'étape précédente </button>
        <div className="btn-group">
          <button disabled={!file} onClick={handleRun}> Lancer la pipeline </button>
          <button disabled={!result} onClick={() => setResult(null)}> Réinitialiser </button>
        </div>
      </div>
    </div>
  );
}
```

```

    return classNames('border-bottom', 'border-radius', 'bg-white');
}

Ministérieller
{
    <button>
        {>}
    </button>
</div>
</div>
(error AA <div className="alert alert-error" style={{ margin:top: 10 }}>& {error}</div>
</div>
</div>

[loading AA {
    <div className="loading-overlay">
        <div className="spinner"> />
        <p>Exécution du pipeline en cours...</p>
    </div>
}]

[result AA {
    <div className="section">
        <div className="section-title">
            Résultats - {result.width}</div>
            {result.height}px - {result.steps.length} étapes
        </div>
        <div className="pipeline-grid">
            <div className="step-card">
                <img src={formData.result.original} alt="Original" />
                <div className="step-info">
                    <span className="step-badge">ORIGINAL</span>
                    <div className="step-name">étape originale</div>
                    <div className="step-desc">avant tout traitement</div>
                </div>
            </div>
            {result.steps.map((step) => {
                <div className="step-card">steps({step.name})
                <img src={formData.step_image}>Alt({step.name}) />
                <div className="step-info">
                    <span className="step-badge">{t.getLabel(step.name)}</span>
                    <div className="step-name">{step.name.replace("Pipeline - /", "")}</div>
                    <div className="step-desc">{step.description}</div>
                </div>
            })
        }
    </div>
</div>
</div>
</div>
]
);
}

```



### 3.3 Détection Classique (Segmentation)

La détection classique utilise le seuillage d'Otsu combiné à des opérations morphologiques (ouverture/fermeture) et la labélisation des composantes connexes pour identifier et compter les objets.

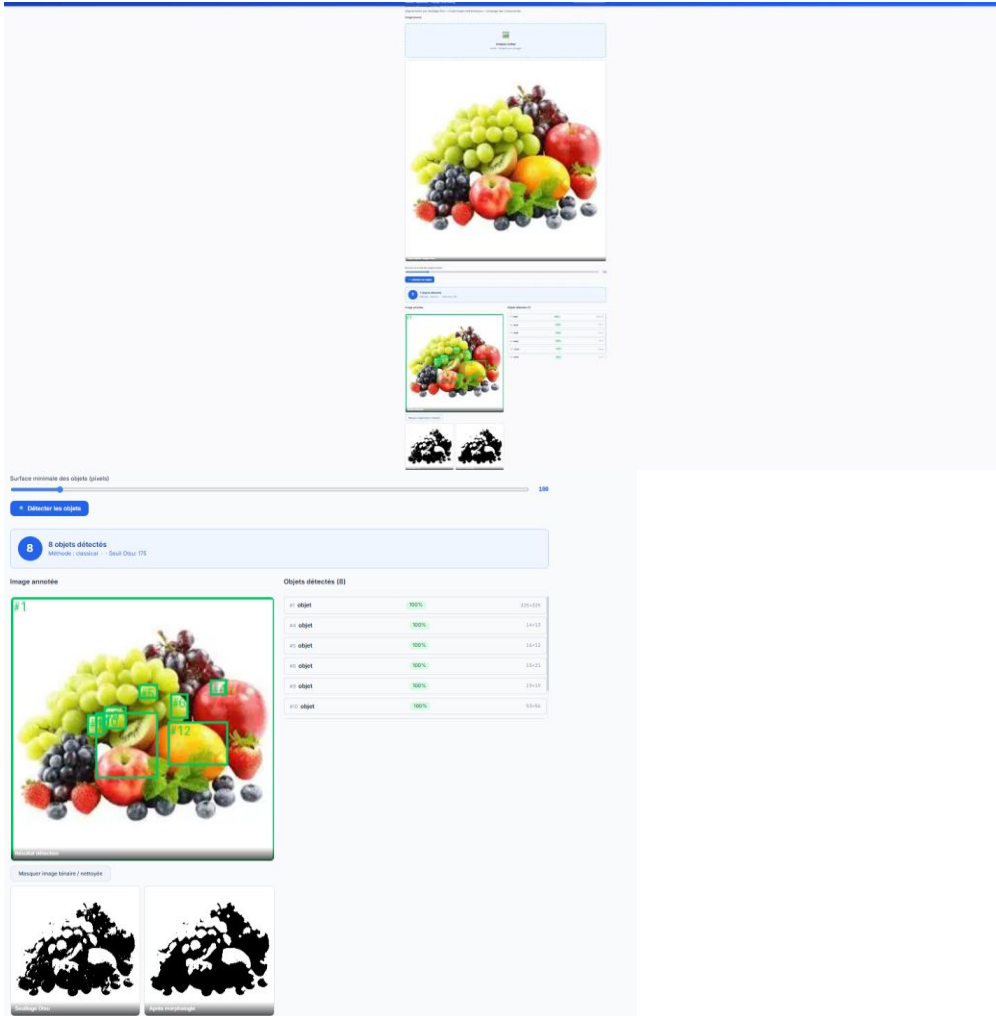


Fig. 5 — Détection classique : 7 objets détectés avec boîtes englobantes

Code source — DetectionPage.tsx :

```
// src/pages/DetectionPage.tsx
```

```

1  # Importer les modules nécessaires
2  import sys
3  import os
4  import random
5  import time
6  import datetime
7  import logging
8  import argparse
9  import json
10 import pickle
11 import hashlib
12 import re
13 import math
14 import itertools
15 import collections
16 import statistics
17 import functools
18 import multiprocessing
19 import concurrent.futures
20 import asyncio
21 import aiohttp
22 import aiofiles
23 import websockets
24 import httpx
25 import requests
26 import urllib3
27 import ssl
28 import socket
29 import socketserver
30 import threading
31 import queue
32 import multiprocessing
33 import concurrent.futures
34 import asyncio
35 import aiohttp
36 import aiofiles
37 import websockets
38 import httpx
39 import requests
40 import urllib3
41 import ssl
42 import socket
43 import socketserver
44 import threading
45 import queue
46 import multiprocessing
47 import concurrent.futures
48 import asyncio
49 import aiohttp
50 import aiofiles
51 import websockets
52 import httpx
53 import requests
54 import urllib3
55 import ssl
56 import socket
57 import socketserver
58 import threading
59 import queue
60 import multiprocessing
61 import concurrent.futures
62 import asyncio
63 import aiohttp
64 import aiofiles
65 import websockets
66 import httpx
67 import requests
68 import urllib3
69 import ssl
70 import socket
71 import socketserver
72 import threading
73 import queue
74 import multiprocessing
75 import concurrent.futures
76 import asyncio
77 import aiohttp
78 import aiofiles
79 import websockets
80 import httpx
81 import requests
82 import urllib3
83 import ssl
84 import socket
85 import socketserver
86 import threading
87 import queue
88 import multiprocessing
89 import concurrent.futures
90 import asyncio
91 import aiohttp
92 import aiofiles
93 import websockets
94 import httpx
95 import requests
96 import urllib3
97 import ssl
98 import socket
99 import socketserver
100 import threading
101 import queue
102 import multiprocessing
103 import concurrent.futures
104 import asyncio
105 import aiohttp
106 import aiofiles
107 import websockets
108 import httpx
109 import requests
110 import urllib3
111 import ssl
112 import socket
113 import socketserver
114 import threading
115 import queue
116 import multiprocessing
117 import concurrent.futures
118 import asyncio
119 import aiohttp
120 import aiofiles
121 import websockets
122 import httpx
123 import requests
124 import urllib3
125 import ssl
126 import socket
127 import socketserver
128 import threading
129 import queue
130 import multiprocessing
131 import concurrent.futures
132 import asyncio
133 import aiohttp
134 import aiofiles
135 import websockets
136 import httpx
137 import requests
138 import urllib3
139 import ssl
140 import socket
141 import socketserver
142 import threading
143 import queue
144 import multiprocessing
145 import concurrent.futures
146 import asyncio
147 import aiohttp
148 import aiofiles
149 import websockets
150 import httpx
151 import requests
152 import urllib3
153 import ssl
154 import socket
155 import socketserver
156 import threading
157 import queue
158 import multiprocessing
159 import concurrent.futures
160 import asyncio
161 import aiohttp
162 import aiofiles
163 import websockets
164 import httpx
165 import requests
166 import urllib3
167 import ssl
168 import socket
169 import socketserver
170 import threading
171 import queue
172 import multiprocessing
173 import concurrent.futures
174 import asyncio
175 import aiohttp
176 import aiofiles
177 import websockets
178 import httpx
179 import requests
180 import urllib3
181 import ssl
182 import socket
183 import socketserver
184 import threading
185 import queue
186 import multiprocessing
187 import concurrent.futures
188 import asyncio
189 import aiohttp
190 import aiofiles
191 import websockets
192 import httpx
193 import requests
194 import urllib3
195 import ssl
196 import socket
197 import socketserver
198 import threading
199 import queue
200 import multiprocessing
201 import concurrent.futures
202 import asyncio
203 import aiohttp
204 import aiofiles
205 import websockets
206 import httpx
207 import requests
208 import urllib3
209 import ssl
210 import socket
211 import socketserver
212 import threading
213 import queue
214 import multiprocessing
215 import concurrent.futures
216 import asyncio
217 import aiohttp
218 import aiofiles
219 import websockets
220 import httpx
221 import requests
222 import urllib3
223 import ssl
224 import socket
225 import socketserver
226 import threading
227 import queue
228 import multiprocessing
229 import concurrent.futures
230 import asyncio
231 import aiohttp
232 import aiofiles
233 import websockets
234 import httpx
235 import requests
236 import urllib3
237 import ssl
238 import socket
239 import socketserver
240 import threading
241 import queue
242 import multiprocessing
243 import concurrent.futures
244 import asyncio
245 import aiohttp
246 import aiofiles
247 import websockets
248 import httpx
249 import requests
250 import urllib3
251 import ssl
252 import socket
253 import socketserver
254 import threading
255 import queue
256 import multiprocessing
257 import concurrent.futures
258 import asyncio
259 import aiohttp
260 import aiofiles
261 import websockets
262 import httpx
263 import requests
264 import urllib3
265 import ssl
266 import socket
267 import socketserver
268 import threading
269 import queue
270 import multiprocessing
271 import concurrent.futures
272 import asyncio
273 import aiohttp
274 import aiofiles
275 import websockets
276 import httpx
277 import requests
278 import urllib3
279 import ssl
280 import socket
281 import socketserver
282 import threading
283 import queue
284 import multiprocessing
285 import concurrent.futures
286 import asyncio
287 import aiohttp
288 import aiofiles
289 import websockets
290 import httpx
291 import requests
292 import urllib3
293 import ssl
294 import socket
295 import socketserver
296 import threading
297 import queue
298 import multiprocessing
299 import concurrent.futures
300 import asyncio
301 import aiohttp
302 import aiofiles
303 import websockets
304 import httpx
305 import requests
306 import urllib3
307 import ssl
308 import socket
309 import socketserver
310 import threading
311 import queue
312 import multiprocessing
313 import concurrent.futures
314 import asyncio
315 import aiohttp
316 import aiofiles
317 import websockets
318 import httpx
319 import requests
320 import urllib3
321 import ssl
322 import socket
323 import socketserver
324 import threading
325 import queue
326 import multiprocessing
327 import concurrent.futures
328 import asyncio
329 import aiohttp
330 import aiofiles
331 import websockets
332 import httpx
333 import requests
334 import urllib3
335 import ssl
336 import socket
337 import socketserver
338 import threading
339 import queue
340 import multiprocessing
341 import concurrent.futures
342 import asyncio
343 import aiohttp
344 import aiofiles
345 import websockets
346 import httpx
347 import requests
348 import urllib3
349 import ssl
350 import socket
351 import socketserver
352 import threading
353 import queue
354 import multiprocessing
355 import concurrent.futures
356 import asyncio
357 import aiohttp
358 import aiofiles
359 import websockets
360 import httpx
361 import requests
362 import urllib3
363 import ssl
364 import socket
365 import socketserver
366 import threading
367 import queue
368 import multiprocessing
369 import concurrent.futures
370 import asyncio
371 import aiohttp
372 import aiofiles
373 import websockets
374 import httpx
375 import requests
376 import urllib3
377 import ssl
378 import socket
379 import socketserver
380 import threading
381 import queue
382 import multiprocessing
383 import concurrent.futures
384 import asyncio
385 import aiohttp
386 import aiofiles
387 import websockets
388 import httpx
389 import requests
390 import urllib3
391 import ssl
392 import socket
393 import socketserver
394 import threading
395 import queue
396 import multiprocessing
397 import concurrent.futures
398 import asyncio
399 import aiohttp
400 import aiofiles
401 import websockets
402 import httpx
403 import requests
404 import urllib3
405 import ssl
406 import socket
407 import socketserver
408 import threading
409 import queue
410 import multiprocessing
411 import concurrent.futures
412 import asyncio
413 import aiohttp
414 import aiofiles
415 import websockets
416 import httpx
417 import requests
418 import urllib3
419 import ssl
420 import socket
421 import socketserver
422 import threading
423 import queue
424 import multiprocessing
425 import concurrent.futures
426 import asyncio
427 import aiohttp
428 import aiofiles
429 import websockets
430 import httpx
431 import requests
432 import urllib3
433 import ssl
434 import socket
435 import socketserver
436 import threading
437 import queue
438 import multiprocessing
439 import concurrent.futures
440 import asyncio
441 import aiohttp
442 import aiofiles
443 import websockets
444 import httpx
445 import requests
446 import urllib3
447 import ssl
448 import socket
449 import socketserver
450 import threading
451 import queue
452 import multiprocessing
453 import concurrent.futures
454 import asyncio
455 import aiohttp
456 import aiofiles
457 import websockets
458 import httpx
459 import requests
460 import urllib3
461 import ssl
462 import socket
463 import socketserver
464 import threading
465 import queue
466 import multiprocessing
467 import concurrent.futures
468 import asyncio
469 import aiohttp
470 import aiofiles
471 import websockets
472 import httpx
473 import requests
474 import urllib3
475 import ssl
476 import socket
477 import socketserver
478 import threading
479 import queue
480 import multiprocessing
481 import concurrent.futures
482 import asyncio
483 import aiohttp
484 import aiofiles
485 import websockets
486 import httpx
487 import requests
488 import urllib3
489 import ssl
490 import socket
491 import socketserver
492 import threading
493 import queue
494 import multiprocessing
495 import concurrent.futures
496 import asyncio
497 import aiohttp
498 import aiofiles
499 import websockets
500 import httpx
501 import requests
502 import urllib3
503 import ssl
504 import socket
505 import socketserver
506 import threading
507 import queue
508 import multiprocessing
509 import concurrent.futures
510 import asyncio
511 import aiohttp
512 import aiofiles
513 import websockets
514 import httpx
515 import requests
516 import urllib3
517 import ssl
518 import socket
519 import socketserver
520 import threading
521 import queue
522 import multiprocessing
523 import concurrent.futures
524 import asyncio
525 import aiohttp
526 import aiofiles
527 import websockets
528 import httpx
529 import requests
530 import urllib3
531 import ssl
532 import socket
533 import socketserver
534 import threading
535 import queue
536 import multiprocessing
537 import concurrent.futures
538 import asyncio
539 import aiohttp
540 import aiofiles
541 import websockets
542 import httpx
543 import requests
544 import urllib3
545 import ssl
546 import socket
547 import socketserver
548 import threading
549 import queue
550 import multiprocessing
551 import concurrent.futures
552 import asyncio
553 import aiohttp
554 import aiofiles
555 import websockets
556 import httpx
557 import requests
558 import urllib3
559 import ssl
560 import socket
561 import socketserver
562 import threading
563 import queue
564 import multiprocessing
565 import concurrent.futures
566 import asyncio
567 import aiohttp
568 import aiofiles
569 import websockets
570 import httpx
571 import requests
572 import urllib3
573 import ssl
574 import socket
575 import socketserver
576 import threading
577 import queue
578 import multiprocessing
579 import concurrent.futures
580 import asyncio
581 import aiohttp
582 import aiofiles
583 import websockets
584 import httpx
585 import requests
586 import urllib3
587 import ssl
588 import socket
589 import socketserver
590 import threading
591 import queue
592 import multiprocessing
593 import concurrent.futures
594 import asyncio
595 import aiohttp
596 import aiofiles
597 import websockets
598 import httpx
599 import requests
600 import urllib3
601 import ssl
602 import socket
603 import socketserver
604 import threading
605 import queue
606 import multiprocessing
607 import concurrent.futures
608 import asyncio
609 import aiohttp
610 import aiofiles
611 import websockets
612 import httpx
613 import requests
614 import urllib3
615 import ssl
616 import socket
617 import socketserver
618 import threading
619 import queue
620 import multiprocessing
621 import concurrent.futures
622 import asyncio
623 import aiohttp
624 import aiofiles
625 import websockets
626 import httpx
627 import requests
628 import urllib3
629 import ssl
630 import socket
631 import socketserver
632 import threading
633 import queue
634 import multiprocessing
635 import concurrent.futures
636 import asyncio
637 import aiohttp
638 import aiofiles
639 import
```

```

2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

### 3.4 Classification IA (MobileNetV2)

La page de classification utilise le modèle MobileNetV2 pré-entraîné sur ImageNet pour classer l'image parmi 1000 catégories. Elle affiche le Top-5 des prédictions avec leurs scores de confiance sous forme de barres de progression.

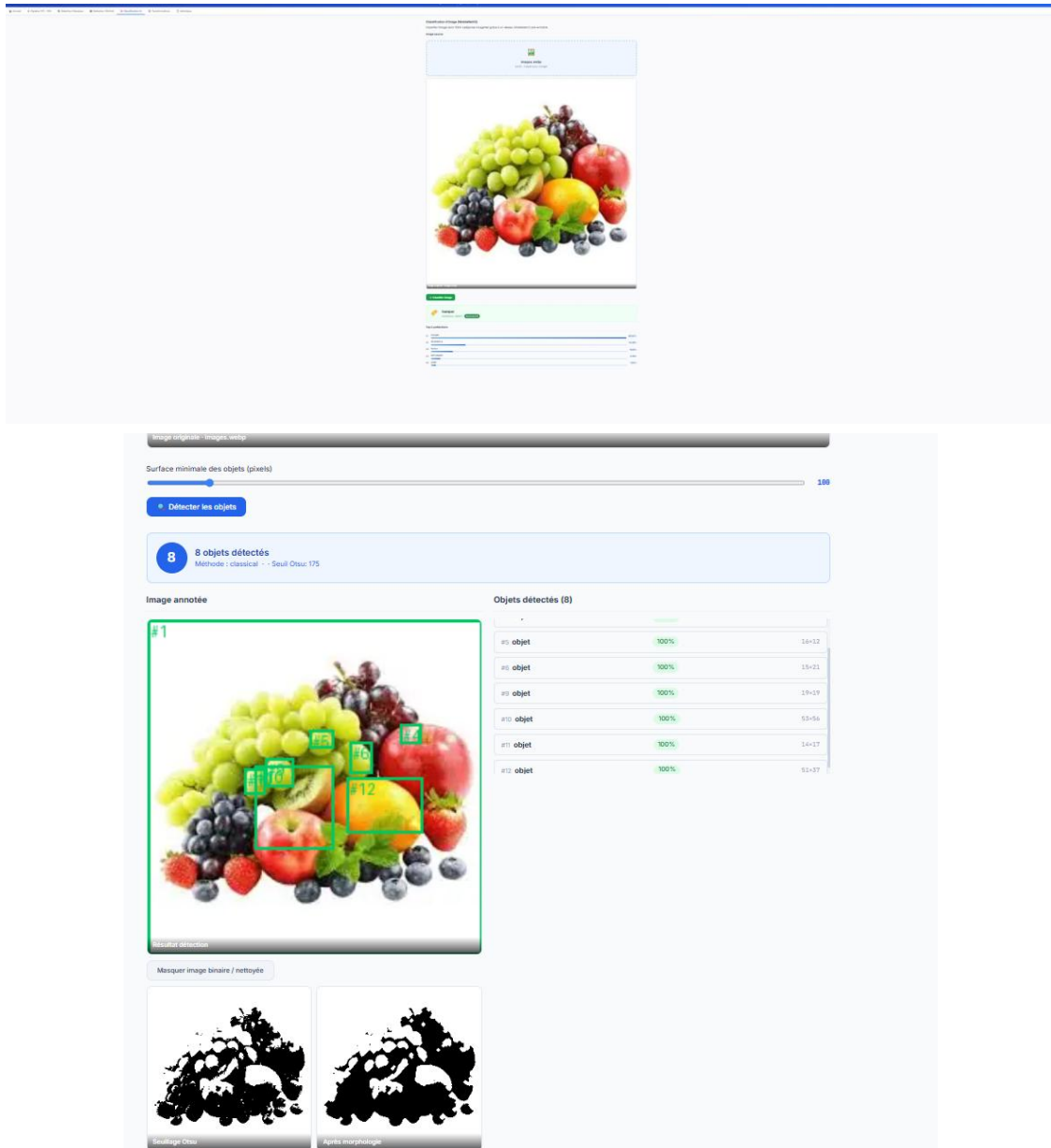


Fig. 6 — Classification MobileNetV2 : Top-5 prédictions avec scores de confiance

Code source — ClassificationPage.tsx :

```
// src/pages/ClassificationPage.tsx
```

```

interface TopClass {
}

Windurf Refactor | Explain

interface ClassResult {
  top_classes: TopClass[];
  model: string;
  error?: string;
}

Tabnine | Edit | Test | Explain | Document | Windurf Refactor | Explain | Generate | JSDoc | X

export default function ClassificationPage() {
  const [file, setFile] = useState<File | null>(null);
  const [preview, setPreview] = useState<string | null>(null);
  const [loading, setLoading] = useState(false);
  const [result, setResult] = useState<ClassResult | null>(null);
  const [error, setError] = useState<string | null>(null);

  Windurf Refactor | Explain | Generate | JSDoc | X
  async function handleClassify() {
    if (!file) return;
    setLoading(true);
    setError(null);
    try {
      const data = await classifyImage(file);
      setResult(data);
    } catch (e: unknown) {
      setError(e instanceof Error ? e.message : 'Erreur');
    } finally {
      setLoading(false);
    }
  }

  const top = result?.top_classes[0];
  const maxConf = result?.top_classes[0]?.confidence ?? 1;

  return (
    <div>
      <div className="section">
        <div className="card-title">Classification d'image (MobileNetV2)</div>
        <p style={{ marginTop: 4, marginBottom: 16 }}>
          Classifie l'image dans 1000 catégories ImageNet grâce à un réseau MobileNetV2 pré-entraîné.
        </p>
        <ImageUploader
          onImageSelected={(f, url) => { setFile(f); setPreview(url); setResult(null); }}
          currentFile={file}
          previewUrl={preview}
        />
        <div className="btn-group">
          <button className="btn btn-success" onClick={handleClassify} disabled={!file || loading}>
            {loading ? '⌛ Classification...' : '🟢 Classifier l\'image'}
          </button>
        </div>
        <div>
          {error && <div className="alert alert-error" style={{ marginTop: 12 }}>⚠️ {error}</div>}
        </div>
      </div>
    </div>
  );
}

```

```
export default function ClassificationPage() {  
    <div className="btn-group">  
        <button className="btn btn-success" onClick={handleClassify} disabled={!file || loading}>  
            {loading ? "🔄 Classification..." : "🔍 Classifier 1\Images"}  
        </button>  
    </div>  
    <div>  
        <error AA <div className="alert alert-error" style={{ marginTop: 12 }}><{ error}</div></div>  
    </div>  
  
    <loading AA {  
        <div className="loading-overlay">  
            <div className="spinner"/>  
            <p>Reference PublisherV2 (ImageNet)...</p>  
        </div>  
    }</div>  
  
    <result AA {  
        <div className="section">  
            <result.error ?>  
                <div className="alert alert-error">& Mobile indisponible.</div></div>  
            <?> {  
                <?>  
                    <top AA {  
                        <div style={{ display: 'flex', alignItems: 'center', gap: 16, marginBottom: 20, padding: '16px 20px', background: 'var(--secondary-50)', border: '1px solid var(--secondary-100)', borderRadius:  
                            <div style={{ fontSzie: '1.2em' }}>#</div>  
                            <div>  
                                <div style={{ fontSzie: '1.2em', fontWeight: 700, color: 'var(--neutral-800)' }}><{top.class}</div>  
                                <div style={{ fontSzie: 0.85em, color: 'var(--secondary-600)', marginTop: 2 }}>  
                                    Confiance : (<{top.confidence * 100}>)<{toFixed(1)}% . <span className="model-badge" style{{ background: 'var(--secondary-700)' }}></span>  
                                </div>  
                            </div>  
                        </div>  
                    </div>  
                </div>  
                <div className="section-title">Top 5 prédictions</div>  
                <div className="classification-list">  
                    <{result.top_classes.map((item, i) => {  
                        <div className="cli-item key-<{item.class}>">  
                            <span className="cli-rank"><{i + 1}>/<{span}>  
                            <div className="cli-bar-wrap">  
                                <div className="cli-name"><{item.class}>/</div>  
                                <div className="cli-bar">  
                                    <div className="cli-bar-fill" style{{ width: "<{(item.confidence / maxConf) * 100}>% }} />  
                                </div>  
                                <div>  
                                    <span className="cli-pct"><{(item.confidence * 100).toFixed(2)}%>/<{span}>  
                                </div>  
                            </div>  
                        </div>  
                    })>  
                </div>  
            }</div>  
        }</div>  
    }</div>  
);
```

### 3.5 Transformations Individuelles

La page Transformations permet d'appliquer et de visualiser en temps réel chacune des 9 transformations disponibles sur une image source. L'utilisateur sélectionne une transformation et voit immédiatement le résultat côte à côte avec l'original.

#### Négatif

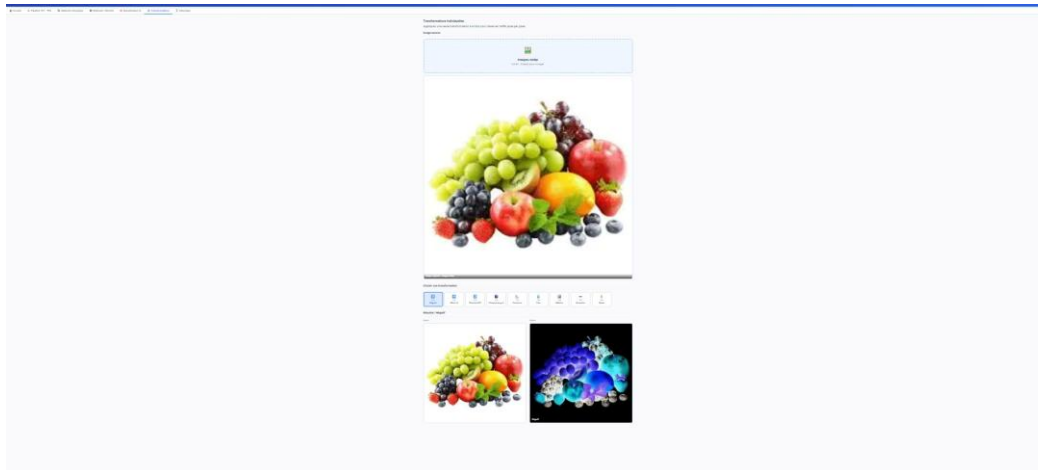


Fig. 7 — Transformation Négatif : inversion des valeurs de pixels (255-p)

#### Miroir Horizontal

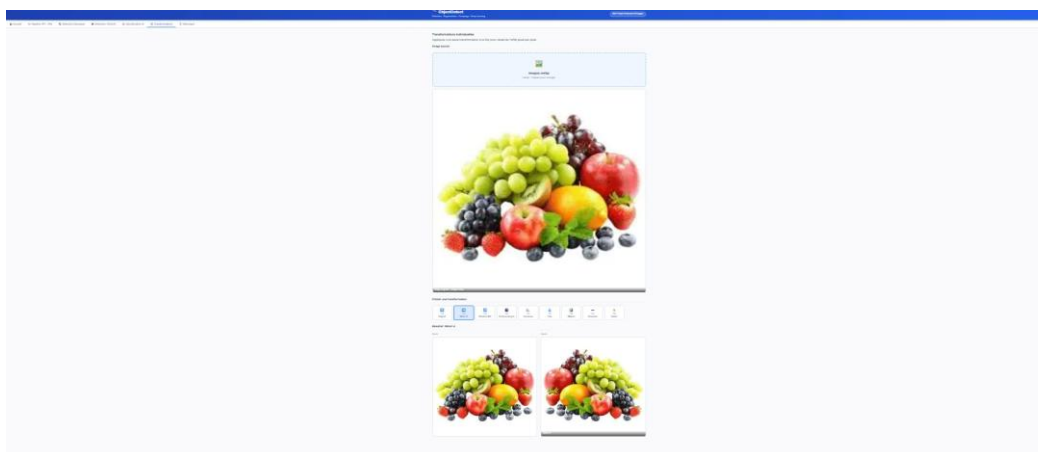


Fig. 8 — Transformation Miroir H : symétrie horizontale

#### Rotation 90°

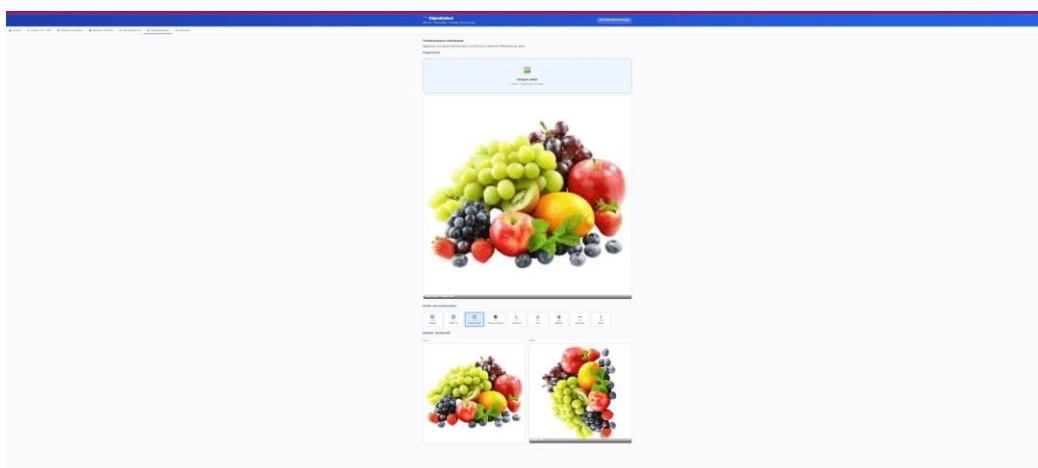


Fig. 9 — Transformation Rotation 90° : pivotement de l'image

## Niveaux de Gris

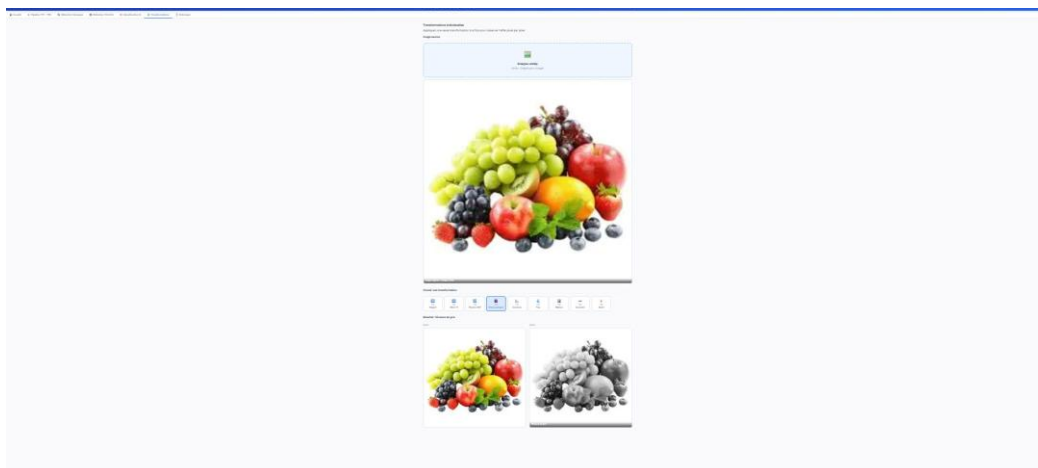


Fig. 10 — Transformation Niveaux de gris : formule luminance BT.601

## Contours (Sobel/Canny)

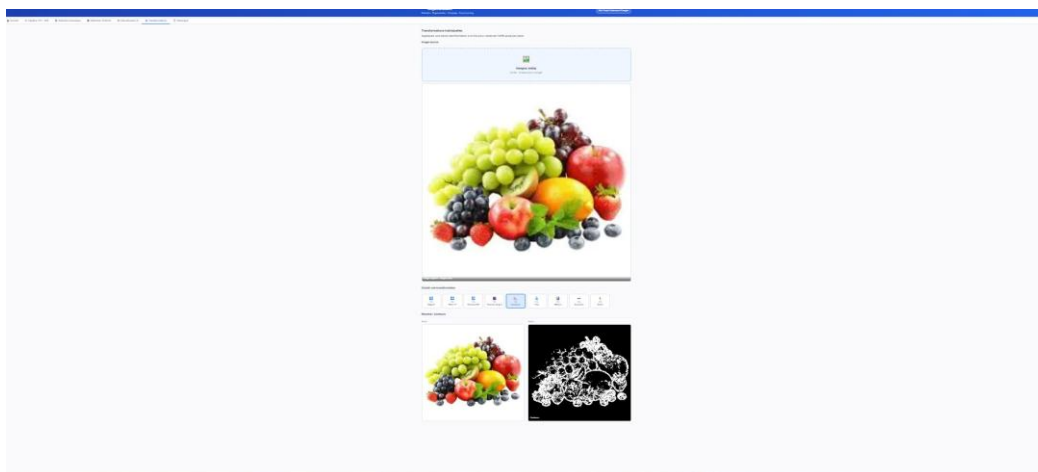


Fig. 11 — Transformation Contours : détection des bords

## Flou

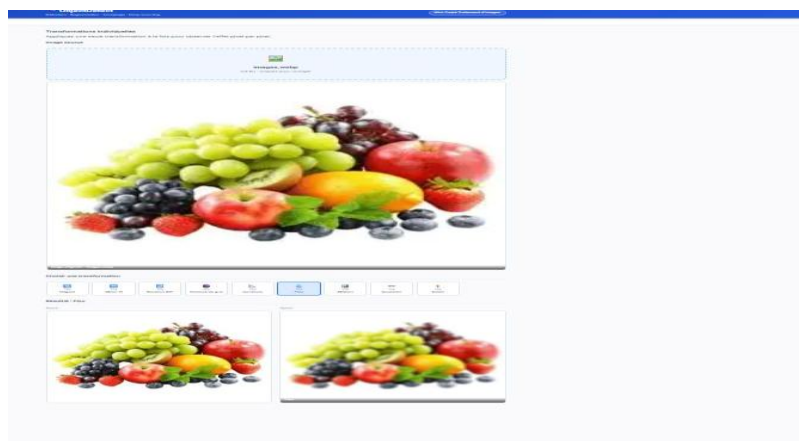


Fig. 12 — Transformation Flou uniforme

## Médian

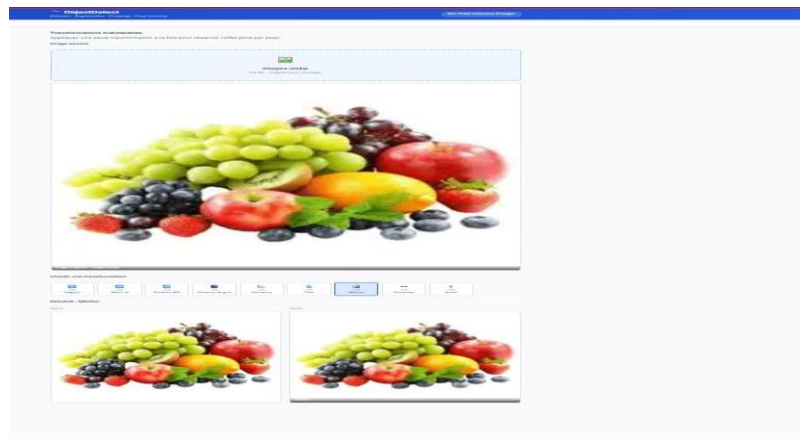


Fig. 13 — Filtre Médian : réduction du bruit

## Gaussien

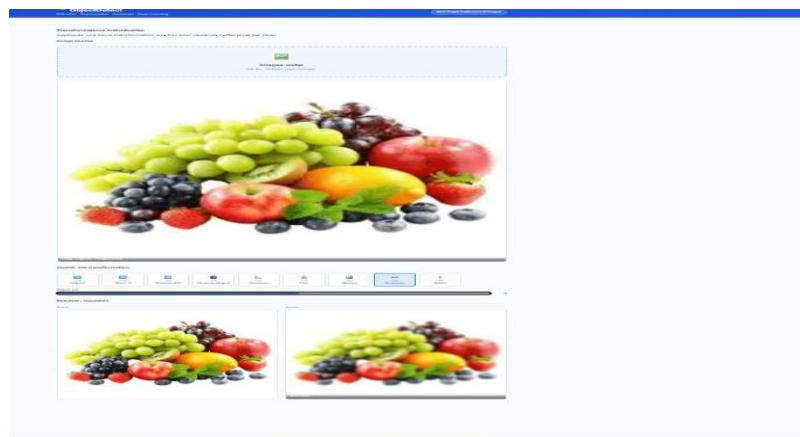


Fig. 14 — Filtre Gaussien : lissage

## Sobel

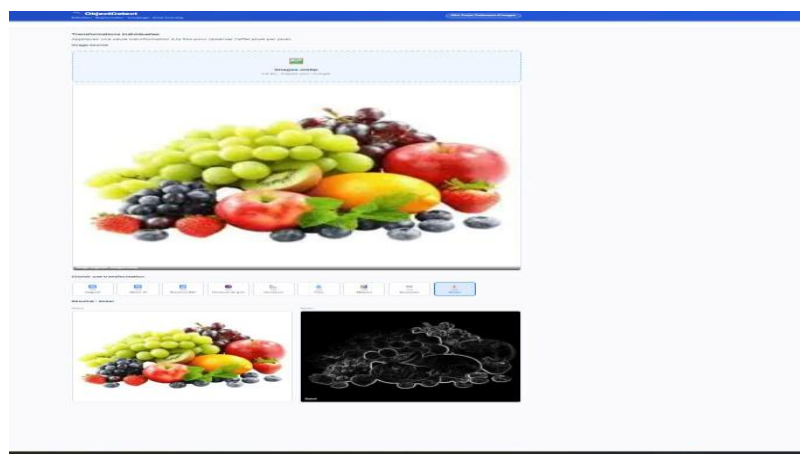


Fig. 15 — Filtre Sobel : gradient  $dx+dy$

Code source — TransformsPage.tsx :

```
// src/pages/TransformsPage.tsx
```

[illegible][illegible]



## 4. Travaux Pratiques Intégrés (TP1–TP6)

Le projet intègre l'ensemble des 6 travaux pratiques du cours de traitement d'images :

|     |                            |                                                                                          |
|-----|----------------------------|------------------------------------------------------------------------------------------|
| TP1 | Manipulation pixel         | Négatif (255-p), symétrie H/V, rotation 90°, comparaison Pillow                          |
| TP2 | Niveaux de gris & contours | Formule luminance BT.601, gradient dx+dy > seuil, comparaison Pillow                     |
| TP3 | Fusion & floutage          | Fusion max canaux RGB, flou uniforme nxn, Pillow BoxBlur                                 |
| TP4 | Filtrage avancé            | Filtre médian, filtre gaussien ( $\sigma$ ), Sobel (convolution), Pillow FIND_EDGES      |
| TP5 | Segmentation & morphologie | Seuillage d'Otsu, érosion/dilatation, ouverture/fermeture, comptage composantes connexes |
| TP6 | Deep Learning              | YOLOv8n COCO (80 classes), MobileNetV2 ImageNet (1000 classes)                           |

### 4.1 Backend — Endpoints FastAPI

Chaque fonctionnalité correspond à un endpoint dédié dans main.py :

```
main.py x requirements.txt start.sh favicon.svg favicon.svg icons.svg icons.svg react.svg
backend > main.py > ...
1  """API FastAPI - Système de Détection et Comptage d'Objets."""
2  import io
3  import base64
4  import os
5  import sys
6  from pathlib import Path
7
8  from fastapi import FastAPI, File, UploadFile, HTTPException, Form
9  from fastapi.middleware.cors import CORSMiddleware
10 from fastapi.responses import JSONResponse
11 from PIL import Image
12 from dotenv import load_dotenv
13
14 load_dotenv(dotenv_path=Path(__file__).parent.parent / ".env")
15
16 sys.path.insert(0, str(Path(__file__).parent))
17
18 from processing.tp1_basic import apply_negative, apply_horizontal_flip, apply_rotation_90
19 from processing.tp2_grayscale import to_grayscale_manual, detect_edges_numpy
20 from processing.tp3_fusion import blur_numpy
21 from processing.tp4_advanced_filters import median_filter_scipy, gaussian_filter_scipy, sobel_filter
22 from processing.tp5_segmentation import threshold_otsu, opening
23 from detection.classical_detector import detect_and_count_classical
24 from detection.deep_detector import detect_with_yolo, classify_image
25
26 SUPABASE_URL = os.getenv("VITE_SUPABASE_URL", "")
27 SUPABASE_ANON_KEY = os.getenv("VITE_SUPABASE_ANON_KEY", "")
28
29 app = FastAPI(title="Image Object Detection API", version="1.0.0")
30
31 app.add_middleware(
32     CORSMiddleware,
33     allow_origins=["*"],
34     allow_credentials=True,
35     allow_methods=["*"],
36     allow_headers=["*"],
37 )
38
39
40 def img_to_base64(img: Image.Image, fmt: str = "PNG") -> str:
41     buf = io.BytesIO()
42     img.save(buf, format=fmt)
43     return base64.b64encode(buf.getvalue()).decode("utf-8")
44
45
46 def load_image(file: UploadFile) -> Image.Image:
47     data = file.file.read()
48     img = Image.open(io.BytesIO(data))
49     return img.convert("RGB")
50
51
```

```

end > mainpy >
@app.get("/health")
def health():
    return {"status": "ok"}

# ===== Pipeline de traitement =====
@app.post("/api/pipeline")
async def run_pipeline(file: UploadFile = File(...)):
    """Pipeline complet: du prétraitement à la détection."""
    try:
        original = load_image(file)
        steps = []

        # TP1: Négatif
        negative = apply_negative(original)
        steps.append(("name": "TP1 - Négatif", "image": img_to_base64(negative), "description": "Inversion des valeurs pixel: 255 - pixel"))

        # TP2: Niveaux de gris
        gray = to_grayscale_manual(original)
        steps.append(("name": "TP2 - Niveaux de gris", "image": img_to_base64(gray), "description": "Formule luminance: 0.299R + 0.587G + 0.114B"))

        # TP2: Détection de contours
        edges = detect_edges_numpy(original, threshold=25)
        steps.append(("name": "TP2 - Détection contours", "image": img_to_base64(edges), "description": "Gradient dx + dy > seuil"))

        # TP3: Floutage
        blurred = blur_numpy(original, kernel_size=5)
        steps.append(("name": "TP3 - Floutage", "image": img_to_base64(blurred), "description": "Filtre uniforme 5x5 sur voisinage"))

        # TP5: Filtre médian
        median = median_filter_scipy(blurred, size=3)
        steps.append(("name": "TP5 - Filtre médian", "image": img_to_base64(median), "description": "Médiane des voisins 3x3 pour éliminer le bruit"))

        # TP5: Filtre Gaussien
        gauss = gaussian_filter_scipy(original, sigma=1.5)
        steps.append(("name": "TP5 - Filtre gaussien", "image": img_to_base64(gauss), "description": "Lissage gaussien 1.5"))

        # TP5: Sobel
        sobel = sobel_filter(gauss)
        steps.append(("name": "TP5 - Filtre Sobel", "image": img_to_base64(sobel), "description": "Détection de contours par convolution Sobel"))

        # TP6: Seuillage Otsu
        binary, otsu_thresh = threshold_otsu(original)
        steps.append(("name": "TP6 - Seuillage Otsu (t={otsu_thresh})", "image": img_to_base64(binary), "description": "Maximisation de la variance inter-classes"))

        # TP6: Morphologie (ouverture)
        opened = opening(binary, iterations=2)
        steps.append(("name": "TP6 - Morphologie (ouverture)", "image": img_to_base64(opened), "description": "Érosion puis dilatation pour nettoyer les objets"))

        return JSONResponse({
            "original": img_to_base64(original),
            "steps": steps,
            "width": original.width,
            "height": original.height,
        })

# ===== Détection classique =====
@app.post("/api/detect/classical")
async def detect_classical(file: UploadFile = File(...), min_area: int = 1000):
    """Détection classique basée sur la segmentation (TP4)."""
    try:
        img = load_image(file)
        result = detect_and_count_classical(img, min_area=min_area)
        return JSONResponse({
            "count": result["count"],
            "objects": result["objects"],
            "annotated": img_to_base64(result["annotated"]),
            "binary": img_to_base64(result["binary"]),
            "cleaned": img_to_base64(result["cleaned"]),
            "threshold": result["threshold"],
            "method": "classical",
        })
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# ===== Détection YOLO =====
@app.post("/api/detect/yolo")
async def detect_yolo(file: UploadFile = File(...), conf: float = 0.25):
    """Détection deep learning avec YOLOv8."""
    try:
        img = load_image(file)
        result = detect_with_yolo(img, conf=threshold=conf)
        return JSONResponse({
            "count": result["count"],
            "objects": result["objects"],
            "class_counts": result.get("class_counts", []),
            "annotated": img_to_base64(result["annotated"]),
            "model": result.get("model", "unknown"),
            "method": "deep_learning",
        })
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# ===== Classification =====
@app.post("/api/classify")
async def classify(file: UploadFile = File(...)):
    """Classification globale de l'image avec MobileNetV2."""
    try:
        img = load_image(file)
        result = classify_image(img)
        return JSONResponse(result)
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

# ===== Transformation =====
@app.post("/api/transform")
async def transform(
    file: UploadFile = File(...),
    operation: str = "neg",
    param: float = 1.5,
):
    """Applique une transformation individuelle (TP1-TP5)."""
    try:
        img = load_image(file)
        ops = {
            "negative": lambda i: apply_negative(i),
            "flip_h": lambda i: apply_horizontal_flip(i),
            "rotation": lambda i: apply_rotation_90(i),
            "grayscale": lambda i: to_grayscale_manual(i).convert("RGB"),
            "edges": lambda i: detect_edges_numpy(i, threshold=10).convert("RGB"),
            "blur": lambda i: blur_numpy(i, kernel_size=5),
            "median": lambda i: median_filter_scipy(i, size=3),
            "gaussian": lambda i: gaussian_filter_scipy(i, sigma=param),
            "sobel": lambda i: sobel_filter(i).convert("RGB"),
        }

        if operation not in ops:
            raise HTTPException(status_code=400, detail=f"Opération inconnue: {operation}")

        output = ops[operation](img)
        return JSONResponse({"image": img_to_base64(output), "operation": operation})
    except HTTPException:
        raise
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

if __name__ == "__main__":
    uvicorn.run(app, host="0.0.0.0", port=8080, reload=True)

```

## 5. Conclusion

---

Ce mini-projet ObjectDetect démontre l'intégration complète des techniques de traitement d'images, depuis les opérations pixel élémentaires (TP1) jusqu'à la détection par deep learning avec YOLOv8 (TP6). L'application est fonctionnelle et couvre l'ensemble du spectre du cours.

### Bilan des compétences acquises

- Traitement d'images classique : filtres, morphologie, segmentation (OpenCV + Pillow)
- Deep Learning appliqué : YOLOv8 pour la détection, MobileNetV2 pour la classification
- Architecture full-stack : React/TypeScript (frontend) + FastAPI/Python (backend)
- Gestion de l'état et des erreurs : résolution des problèmes PostCSS et CORS
- Persistance des données : intégration Supabase pour l'historique des analyses